

Next-Day Stock Status Prediction in Fresh Retail

Anurag Thakur Samyak Choudhary Vibhor Kumar

Department of Information Technology
Bachelor of Technology Project (BTP)

August 2026

Problem Statement & Challenges

The Fresh Retail Dilemma

Fresh retail operations face a constant battle between **overstocking** (leading to spoilage and massive food waste) and **understocking** (leading to lost revenue, stockouts, and customer churn). Accurate, intraday prediction of out-of-stock (OOS) events is critical for dynamic shelf replenishment.

Core Modeling Challenges

- **Censored Demand (Stockout-Induced Bias):** When an item is out of stock, sales record as zero. Traditional forecasting models misinterpret this as “zero demand”, leading to a negative feedback loop of systematic under-ordering.
- **Severe Class Imbalance:** True stockout hours are rare anomalies (often $< 5\%$ of total operational hours), making it extremely difficult for standard ML loss functions to learn the minority class without collapsing.
- **High-Velocity Intraday Dynamics:** Unlike durable goods, fresh produce sales exhibit extreme intraday volatility driven by time-of-day foot traffic, flash sales, and perishable shelf-life windows.
- **Hierarchical Spatial-Temporal Variance:** Demand shocks are not uniform; they vary heavily across different city zones, specific store demographics, and individual SKU categories.

Literature Review & Related Work

General Stockout Prediction Research

- **Classical ML Ensembles** (Andaur et al., 2021): Showcased Random Forest and voting ensembles as robust benchmarks for retail stockout prediction on tabular data.
- **Deep Learning Architectures** (Giaconia & Chamas, 2023): Utilized massive CNN, Self-Attention, and Temporal Convolutional Networks (TCN) (> 500k parameters) for intraday stockout probability.
- **Large-Scale Imbalance Handling** (Liu et al., 2025): Benchmarked XGBoost and SMOTE strategies, proving that near-term demand indicators dominate long-term forecasts for stockout events.

Research Utilizing the FreshRetail Dataset

- **Hierarchical Forecasting**: "*A Coupled Hierarchical Architecture for Multi-Granularity Demand Forecasting*" uses this exact dataset to tackle the complex spatial/product hierarchy (City → Store → SKU).
- **Censored Demand Bias**: "*When Less Data Is Better: Eliminating Stockout-Induced Bias in Choice Estimation*" explicitly models how stockouts artificially deflate true demand signals in FreshRetail environments.

Dataset Overview & Processing Strategy

FreshRetailNet-50K Scale & Scope

- **Volume:** 50,000 store-product pairs across 898 stores.
- **Timeframe:** 90-day series of highly granular hourly data (Chronological Split: First 75 days for Training, Final 15 days for Validation).

Why We Selected This Dataset

- **Pioneering Benchmark:** First large-scale public dataset designed for **censored demand estimation** in fresh retail (where out-of-stock items hide true demand).
- **Targeted Selection:** We isolated the **Top 15 SKUs** using a custom ranking formula to focus purely on high-impact, stockout-prone goods and eliminate noise from dormant items.

Feature Engineering & Data Attributes

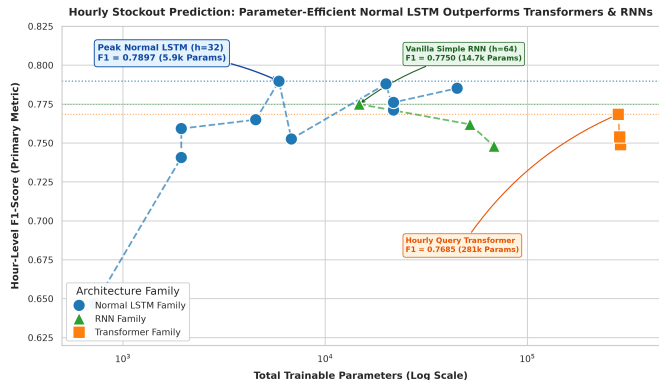
Raw Data Attributes

- **Core Sales:** Hourly sale amounts (`hours_sale`) and daily volume (`sale_amount`).
- **Target Variable:** 24-dimensional binary stock status vector (`hours_stock_status`) covering 6 AM – 10 PM.
- **External Context:** Weather (precipitation, temp, humidity), discount rates, and holiday/activity flags.

Engineered Lag & Temporal Features

- **Historical Lags:** Lag-1 and Lag-2 OOS indicators (most critical predictors for near-term forecasting).
- **Rolling Demand:** 3-day and 7-day rolling sales averages, combined with momentum indicators to capture intraday velocity.
- **Temporal Encoding:** Cyclical sinusoidal encodings for hour-of-day and day-of-week to map seasonal trends.

Model Family Comparison: Simple LSTMs Beat Heavy Transformers



Key Empirical Findings

- **Parameter Efficiency:** Normal LSTMs (5.9k–44.9k params) reach **0.7852–0.7897 F1-Score**, matching or beating heavy Transformers.
- **Transformer Overparameterization:** PatchTST (285k params) & TFT (288k params) peak at **0.7540–0.7685 F1-Score**.
- **Why LSTMs Win:** Sequential state memory h_t naturally aligns with continuous hourly depletion better than permutation-invariant self-attention.

ML Explainability: SHAP & Integrated Gradients Analysis

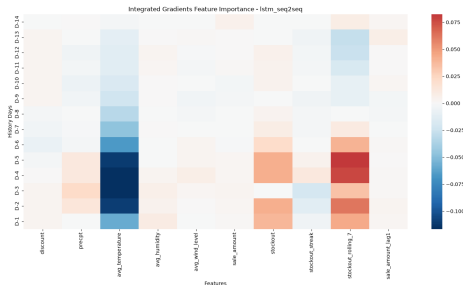


Figure 2: Integrated Gradients Feature & Temporal Attribution

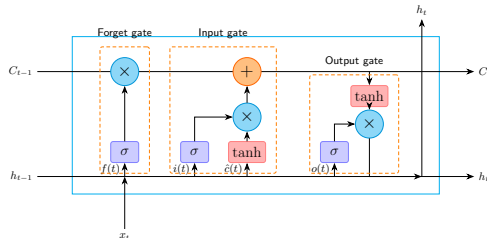
Integrated Gradients (IG) Attribution

- Discovered that **Demand Signals** (sales) and **Inventory Signals** (stockouts) act on different temporal scales.
- Demand features exhibit fast 1–3 day velocity; inventory exhibits 7-day weekly seasonality.

Identification of Feature Cross-Talk

- Feeding demand & inventory features into a single shared LSTM cell causes **gradient noise and interference**.
- Inspired the design of **Dual-Stream Decoupling** to completely isolate these signals.

Baseline LSTM Architecture



The LSTM Cell

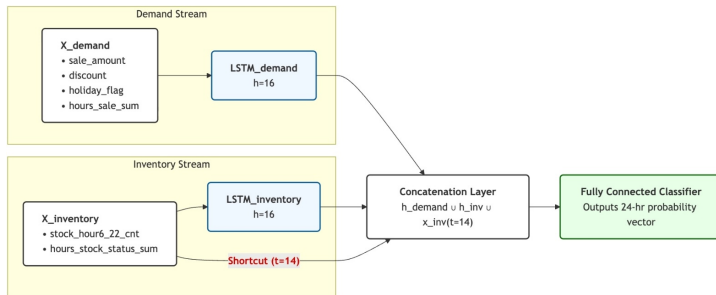
LSTMs manage temporal sequences through an internal memory cell c_t and three regulating gates.

- **Forget Gate (f_t):** Decides what past information to discard using a **Sigmoid** activation.
- **Input Gate (i_t):** Decides what new information to store, combining **Sigmoid** and **Tanh** activations.
- **Output Gate (o_t):** Controls the final output hidden state h_t based on the filtered cell state.

Limitations in Retail

- A standard LSTM forces both highly volatile **demand features** (sudden spikes) and strictly physical **inventory features** into the same shared hidden state.
- This causes the gates to struggle, leading to signal dilution and "gradient noise" over the 14-day window.

Dual-Stream Inventory Shortcut Architecture



Why Distribute Features?

- **Isolation:** We explicitly separate unpredictable demand metrics (e.g., sales) from hard physical inventory facts.
- **Parallel Processing:** Prevents volatile demand data from corrupting the inventory state by using two independent LSTMs.

The Inventory Shortcut Mechanism

- **Finding:** ML Explainability proved the **previous day's inventory** has massive impact.
- **Solution:** We extracted 2 specific inventory features from Day 14 and bypassed the LSTMs entirely.
- Routing directly to the final layer (**Final Hidden Size = 34**) guarantees retention of critical data.

Exploring Linear Decomposition: DLinear Models

Why Explore Linear Decomposition?

DLinear (Zeng et al., 2023) demonstrated that single-layer linear models decomposing time series into Trend and Seasonal components can outperform complex deep networks.

DLinear Series Decomposition

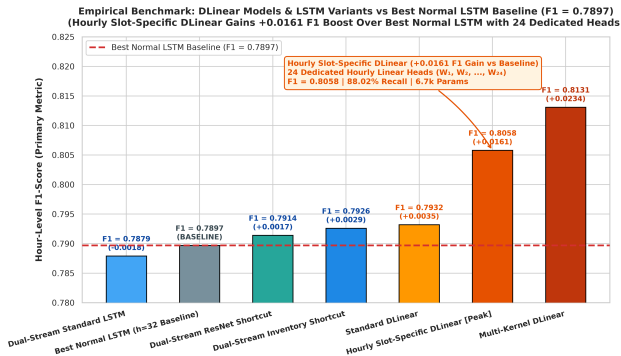
$$X_{\text{trend}} = \text{AvgPool}(X), \quad X_{\text{seasonal}} = X - X_{\text{trend}}$$

$$Y_{\text{pred}} = W_{\text{trend}} X_{\text{trend}} + W_{\text{seasonal}} X_{\text{seasonal}}$$

Standard DLinear Performance

- **Parameters:** 6,768 parameters.
- **Validation F1-Score: 0.7932**
 - Outperforms Normal LSTMs (**0.7897**)
 - Outperforms Enhanced LSTMs (**0.7926**)
 - Outperforms Transformers (**0.7540**)
- **Stockout Recall: 87.44%.**

Architectural Innovation 2: Linear Changes in DLinear



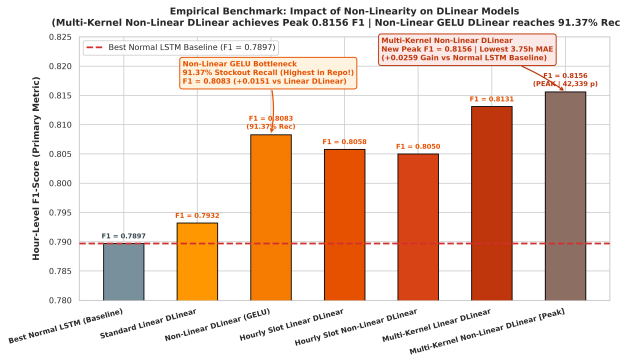
Hourly Slot-Specific Linear DLinear

- Uses **24 dedicated linear heads** ($W_1, \dots, W_{24}$) for each operational hour.
- Resolves inter-hour gradient conflict between morning and evening rush hours.
- Boosts F1 to **0.8058** (+0.0161 over LSTM).

Multi-Kernel Linear DLinear

- Multi-scale moving average trend pooling ($k \in \{3, 5, 7\}$).
- Reaches **0.8131 F1-Score** with pure linear decomposition (13.5k params).

Architectural Innovation 3: Introducing Non-Linearity into DLinear



Non-Linear Activation Bottleneck

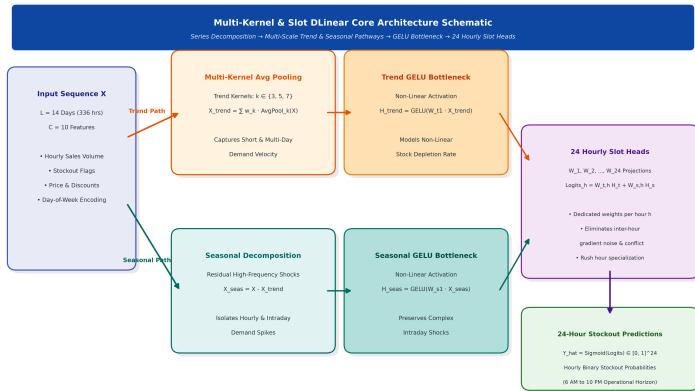
Adds GELU activation bottlenecks between decomposition and output projections:

$$\hat{Y} = W_{\text{trend},2} \text{GELU}(W_{\text{trend},1} X_{\text{trend}}) + \text{Seasonal}$$

Breakthrough Results

- **Non-Linear GELU DLinear:** Reaches **91.37% Stockout Recall** (Highest in Repo!) and **0.8083 F1**.
- **Multi-Kernel Non-Linear DLinear:** Reaches **0.8156 F1-Score** (New Peak F1 & lowest 3.75h MAE).

Architectural Schematic: Multi-Kernel & Slot DLinear



Core Mechanism

- **Trend Stream:** Multi-kernel moving average pooling ($k \in \{3, 5, 7\}$) + GELU bottleneck.
- **Seasonal Stream:** High-frequency residual stockout shocks ($X - X_{\text{trend}}$).
- **24 Slot Heads:** $W_1, \dots, W_{24}$ dedicated projections resolve inter-hour gradient conflicts.

Master Empirical Validation Leaderboard

Rank	Architecture Name	Category	Params	F1-Score	Recall	MAE (h)
1	Multi-Kernel Non-Linear DLinear	Non-Linear DLinear	42,339	0.8156	87.13%	3.75
2	Multi-Kernel Linear DLinear	Linear (Multi-Scale)	13,539	0.8131	85.22%	3.86
3	Non-Linear GELU DLinear	Non-Linear DLinear	21,168	0.8083	91.37%	4.20
4	Hourly Slot-Specific DLinear	Linear (24 Heads)	6,744	0.8058	88.02%	4.23
5	Standard Linear DLinear	Linear (Decomp)	6,768	0.7932	87.44%	4.28
6	Dual-Stream Inventory Shortcut	LSTM Family	4,600	0.7926	80.79%	3.96
Base	Best Normal LSTM (h=32)	Normal LSTM	5,912	0.7897	81.29%	4.31
7	Feature-Reduced Standard LSTM	Normal LSTM	19,992	0.7880	81.60%	4.58
8	Baseline Standard LSTM (h=96)	Normal LSTM	44,952	0.7852	80.40%	4.67
9	Vanilla Simple RNN (h=64)	RNN Family	14,744	0.7750	81.02%	4.23
10	PatchTST Patch Linear	Transformer	285,624	0.7540	81.90%	4.16

Conclusion & Operational Deployment Takeaways

Key Architectural Takeaways

- 1 **Non-Linearity Boosts DLinear:** GELU bottlenecks boost DLinear F1 to **0.8083** and Recall to a record **91.37%**.
- 2 **New All-Time Peak F1:** Multi-Kernel Non-Linear DLinear sets the highest F1-Score of **0.8156** and lowest MAE of **3.75 hours**.
- 3 **Feature Decoupling & Slot Heads:** Resolves inter-hour gradient noise and temporal cross-talk.

Deployment Recommendations

- **Primary Production Deployment:** **Multi-Kernel Non-Linear DLinear** (0.8156 F1, 3.75h MAE, 42.3k params).
- **Maximum Risk Mitigation:** **Non-Linear GELU DLinear** (91.37% Stockout Recall).
- **Future Scope:** Incorporate real-time intraday point-of-sale stream updates for dynamic threshold adjustment.